

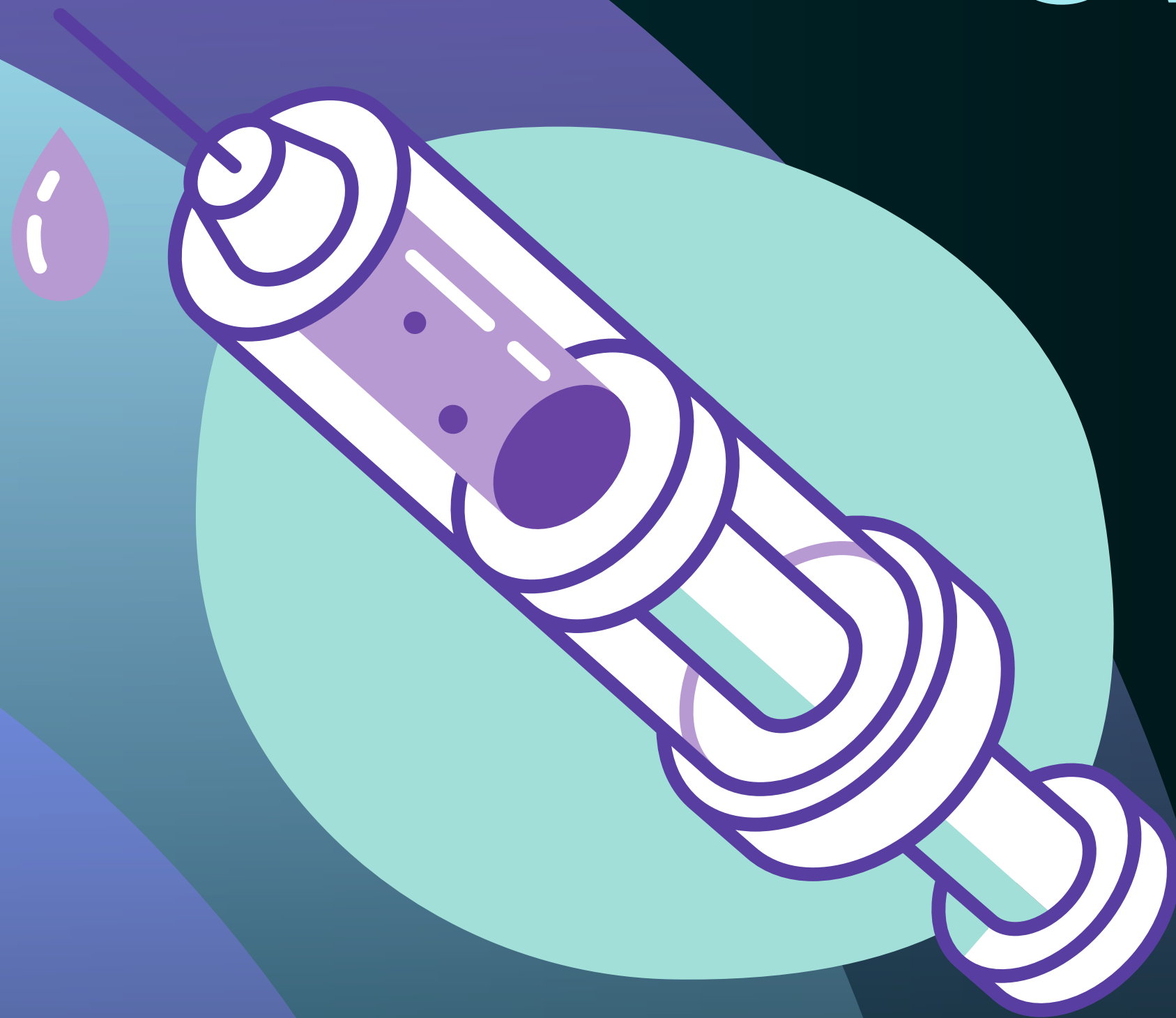


Zainab Lorgat

OWASP TOP 10

A05:2025

INJECTION





**100% of applications tested
had some form of injection
testing**

37 CWEs mapped

62,445 CVEs total

**1.4+ million total
occurrences**



BACKGROUND

Injection moved from #3 to #5 in OWASP Top 10 (2025), but remains one of the most critical and most tested categories.

- Includes:
 - Cross-Site Scripting (XSS) → High frequency, lower average impact (~30k CVEs)
 - SQL Injection (SQLi) → Lower frequency, very high impact (~14k CVEs)

Because XSS has many reported CVEs with lower average impact, it reduces the overall weighted impact score of the category.



WHAT IS INJECTION?

An injection vulnerability occurs when **untrusted user input** is sent to an interpreter and executed as code or commands instead of treated as data.





WHEN IS AN APPLICATION VULNERABLE?

Unsanitized data is passed
into ORM search parameters

User input is not validated,
filtered, or sanitized

Hostile input is directly
executed in stored
procedures

Dynamic queries are built
using string concatenation

Non-parameterized database
queries are used

User input controls query
structure (e.g., table names,
columns)



HOW TO DETECT?

Common injection types include SQL, NoSQL, OS command, ORM, LDAP, and EL/OGNL injection, but the underlying concept is the same: **untrusted input is interpreted and executed as code.**

Effective detection requires a combination of secure code review and automated testing, including fuzzing of all inputs such as parameters, headers, URLs, cookies, JSON, SOAP, and XML. Integrating SAST, DAST, and IAST tools into the CI/CD pipeline further helps identify injection flaws before deployment.

A related modern risk is LLM prompt injection, addressed separately in the OWASP LLM Top 10 (LLM01:2025).



PREVENTING INJECTION

**“ALWAYS SEPARATE DATA
FROM COMMANDS.”**

→
EVEN PARAMETERIZED STORED PROCEDURES CAN BECOME
VULNERABLE IF THEY:

- CONCATENATE STRINGS
- USE EXECUTE IMMEDIATE
- USE EXEC()
- BUILD DYNAMIC SQL INTERNALLY

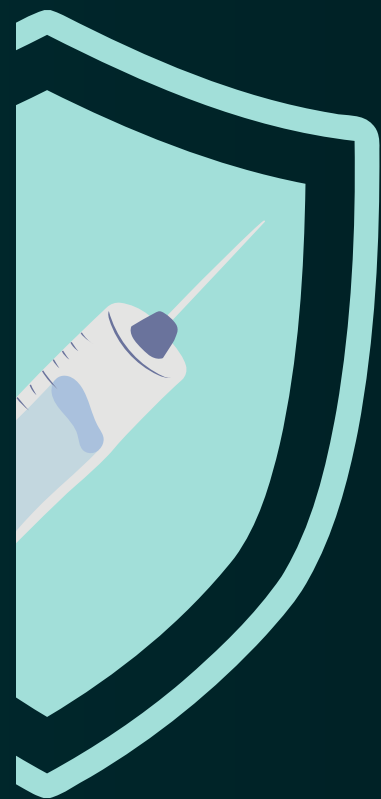
Preferred option:

- Use a safe API
- Use parameterized queries (prepared statements)
- Use properly configured ORM tools
- Avoid direct interpreter usage



PREVENTING INJECTION

**WHEN SEPARATION IS NOT
POSSIBLE (RISK REDUCTION ONLY)**



Positive server-side input validation

- Whitelist expected patterns
- Reject unexpected characters
- Not complete protection

Context-aware escaping

- Escape special characters based on interpreter
- SQL escaping for SQL
- HTML encoding for browser output
- Cannot escape structural elements (table names, column names)

CONCLUSION

Injection turns simple user input into dangerous commands. When applications mix data with code, attackers can take control. The fix is simple but powerful: always separate data from commands using parameterized queries or safe APIs — secure design stops injection before it starts.

